

# where does a message start?

digitalisierung und programmierung · input, session 9

## agenda

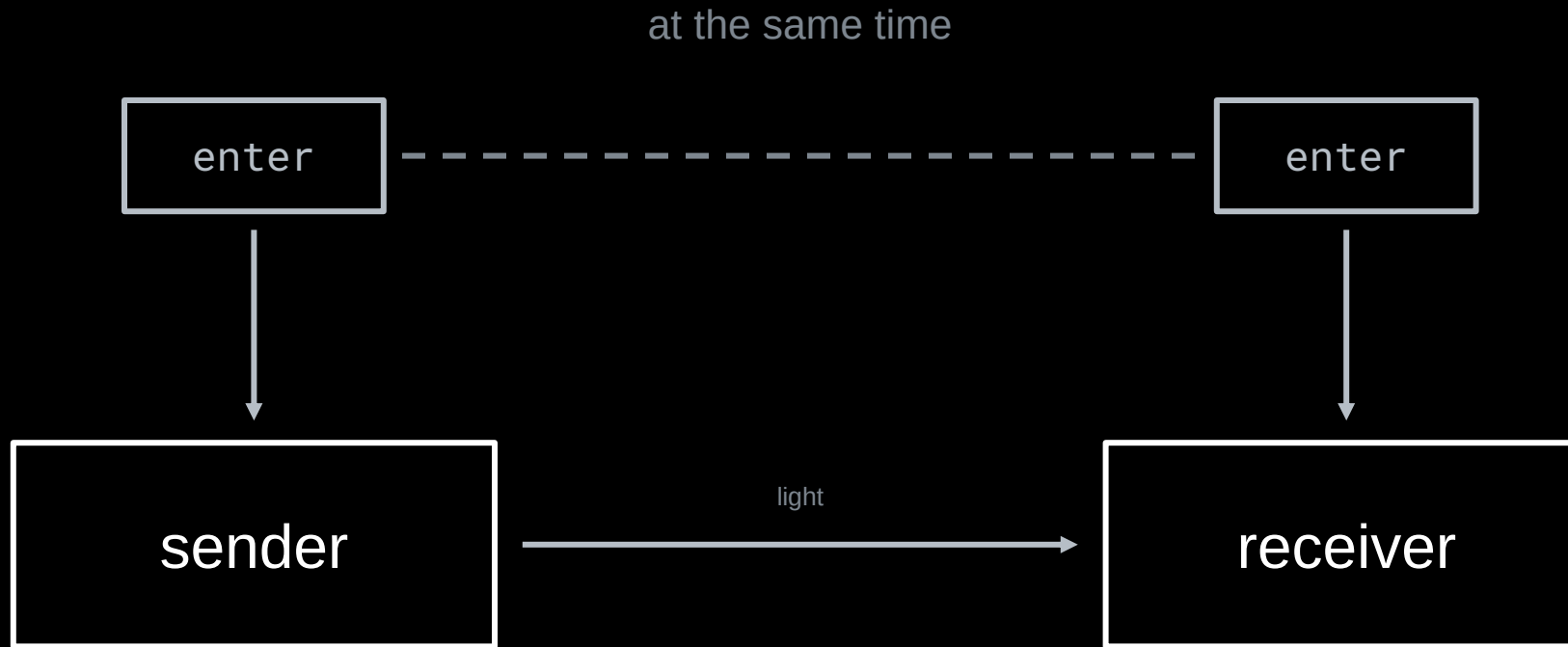
- 1 who says "now"?
- 2 the listener
- 3 the end
- 4 the agreement

who says "now"?

mayday,  
mayday,  
mayday



so far, a human said "now"

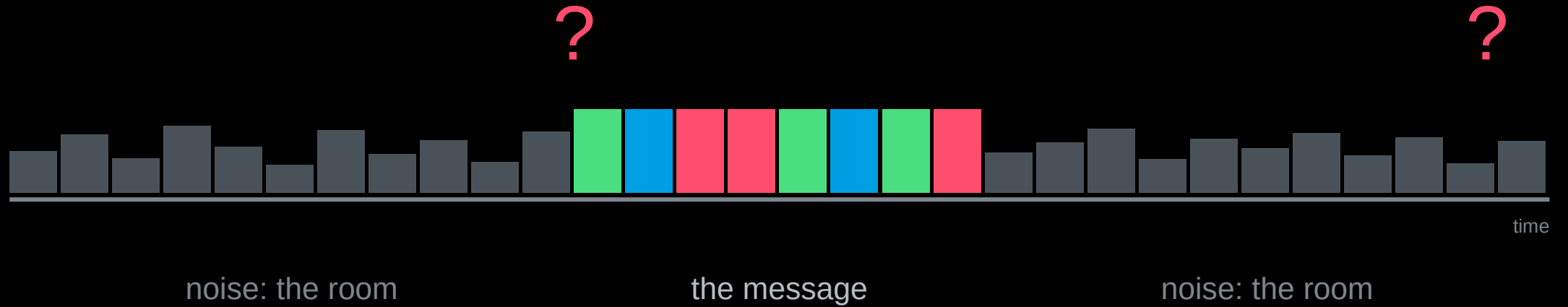


challenge 1 and 2

someone pressed enter on both machines. that was the frame.

1 who says "now"?

the receiver hears the room all the time



where does it start? where does it end? the numbers do not say.

a bit stream without a frame means nothing

what a running receiver needs

where it starts



what a running receiver needs

where it starts

where it ends

what a running receiver needs

where it starts

where it ends

what to do when something goes wrong

the listener

round one

i send. you write down what you see.

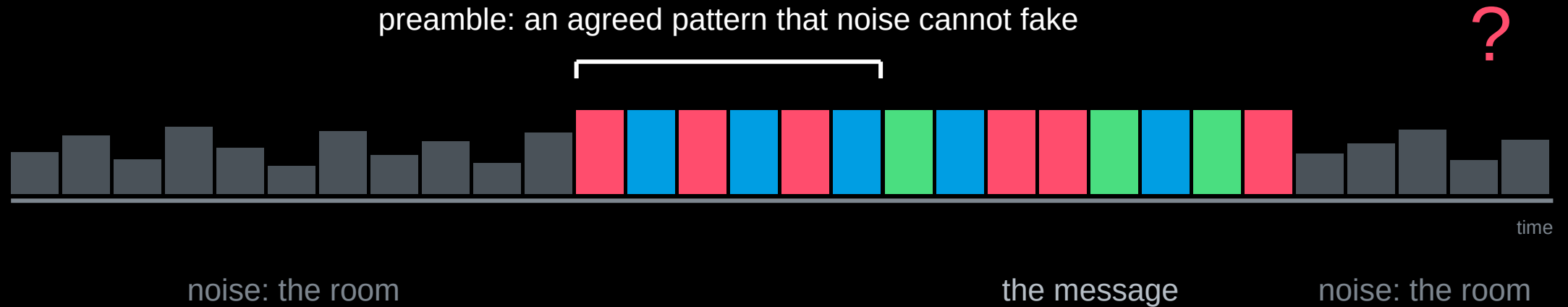
no talking, no hints.

round two

same game. three times.

what changed?

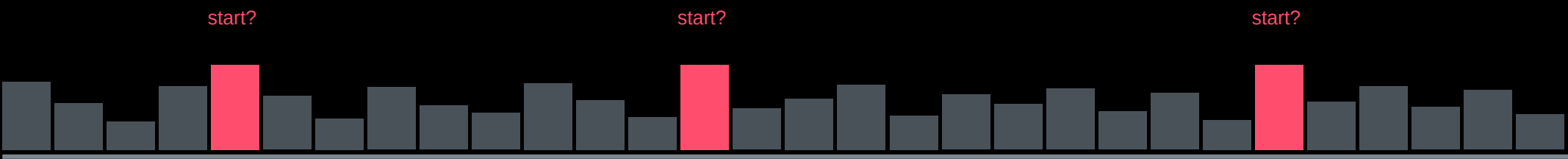
you just invented the preamble



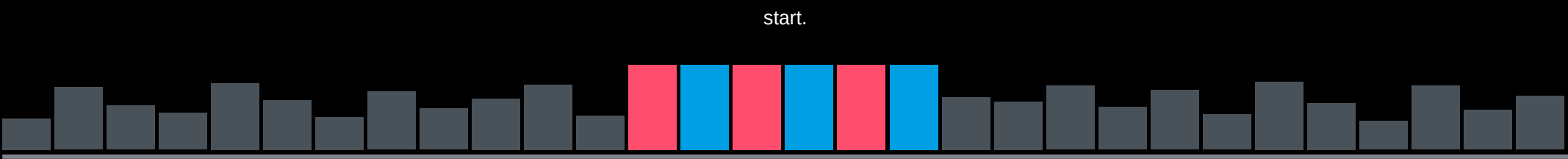
a preamble carries no information. it only says „from here on“.

one red is not a start

start pattern: a single red

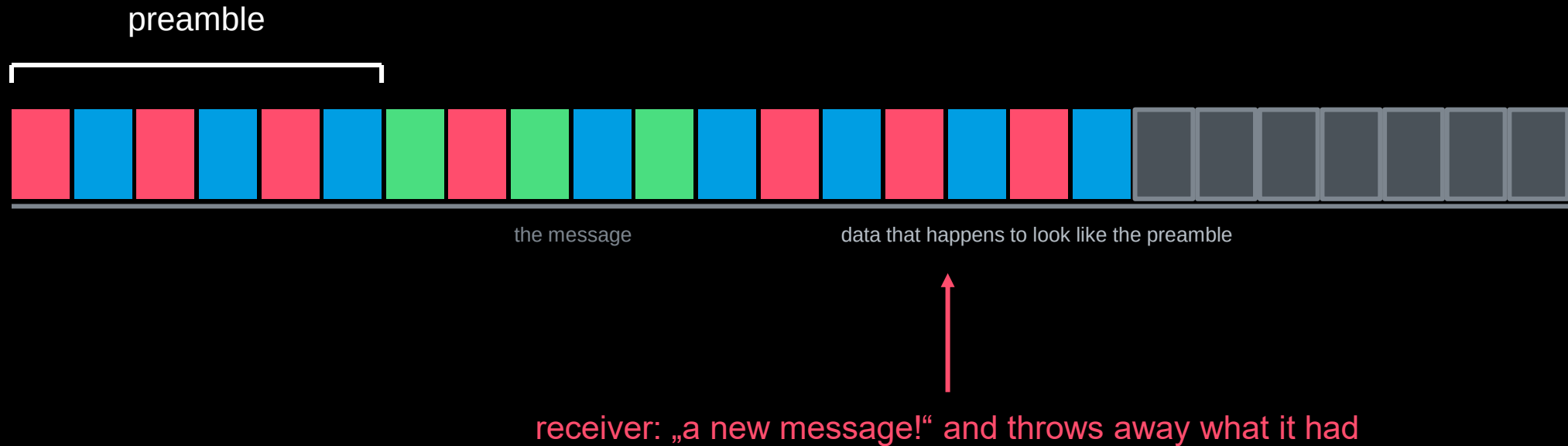


start pattern: red, blue, red, blue, red, blue



longer and more regular: rarer by chance. price: a few symbols per message.

what if the data looks like the start?



your specification must answer this. there is more than one good answer.



the end

round three

# two messages, back to back.

where does the first one stop?

two answers, two price tags

## length field

„12 characters follow“

the receiver counts along

must arrive intact

limits the length

two answers, two price tags

## length field

„12 characters follow“

the receiver counts along

must arrive intact

limits the length

## end marker

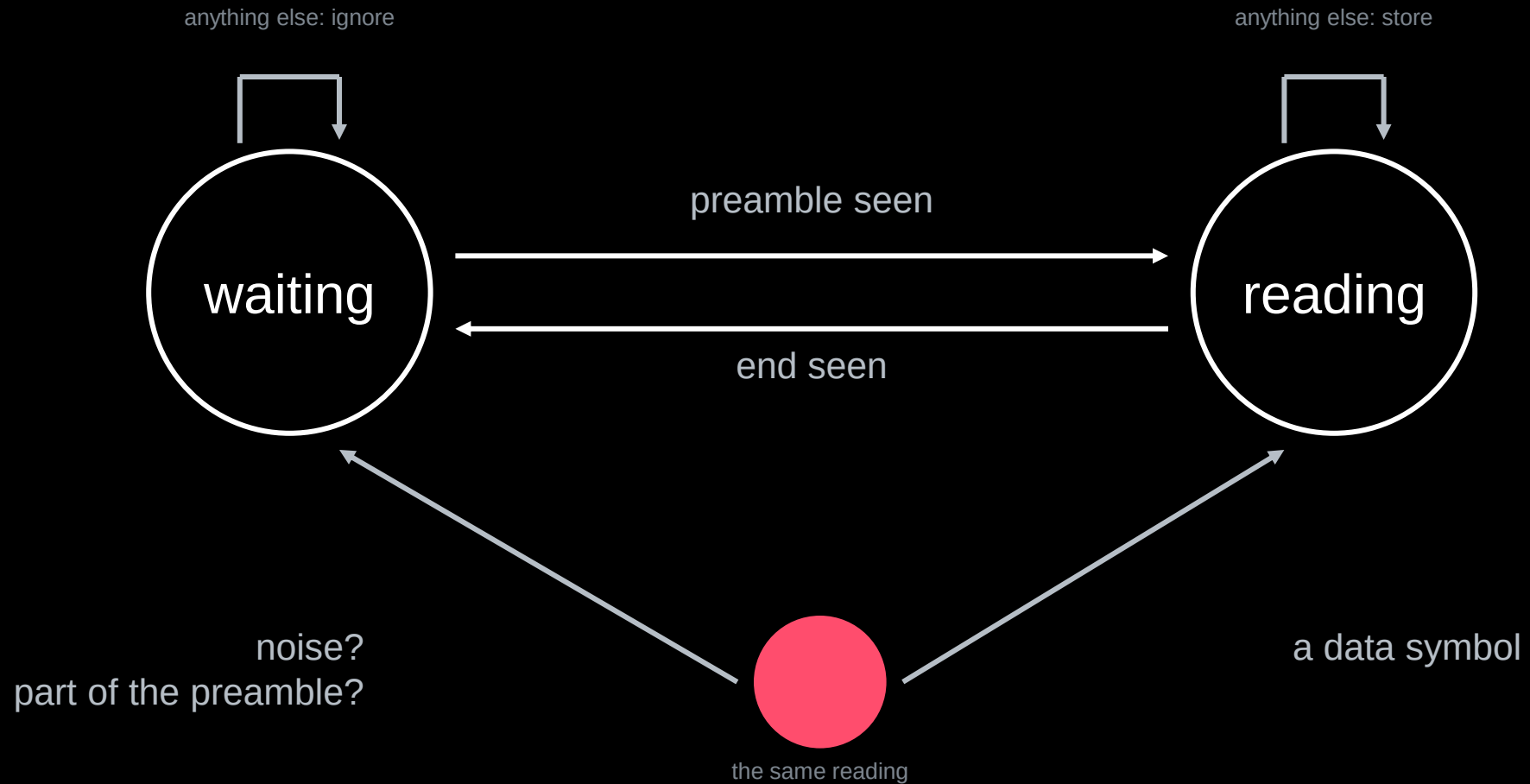
„stop“

any length

must never appear in the data

neither is the right one. you pick one and write it down.

the same reading means different things



the reading does not decide. the state does.

```
state = "waiting"
while True:
    symbol = read_symbol()
    if state == "waiting":
        if preamble_complete(symbol):
            state = "reading"
    elif state == "reading":
        if symbol == END_MARK:
            print(message)
            state = "waiting"
        else:
            message = message + symbol
```

# the agreement

a protocol is an agreement  
someone else can implement



the test for your specification

cross out everything you know  
but did not write down.

is the rest enough?

the classic gap: time

what happens next

this week: your own frame, written down

what happens next

this week: your own frame, written down

next week: someone else builds your receiver

what happens next

this week: your own frame, written down

next week: someone else builds your receiver

then: one frame for the whole class